

```
# =====
# Spark for Machine Learning - Price Prediction
# Linear Regression on Airbnb Dataset
# =====

# Step 0: Install and Start Spark

!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Airbnb Price Prediction") \
    .getOrCreate()

print("Spark started successfully")
```

Spark started successfully

```
from google.colab import files
```

```
uploaded = files.upload()
```

airbnb_pric..._sample.csv

airbnb_price_prediction_sample.csv(text/csv) - 8360 bytes, last modified: 5/9/2026 - 100% done
Saving airbnb_price_prediction_sample.csv to airbnb_price_prediction_sample.csv

```
df = spark.read.csv(
    "airbnb_price_prediction_sample.csv",
    header=True,
    inferSchema=True
)

df.show(5)
```

id	bathrooms	bedrooms	beds	accommodates	minimum_nights	number_of_reviews	review_scores_rating	property_type	room_type
1	1.0	1.0	2.0	3	2	57	74.2	Apartment	Shared room
2	3.0	1.0	1.0	2	2	59	85.2	Apartment	Shared room
3	3.0	1.0	1.0	4	3	207	96.1	House	Shared room
4	1.0	1.0	1.0	2	1	97	72.9	Private room	Shared room
5	4.0	3.0	3.0	7	1	141	78.8	Studio	Private room

only showing top 5 rows

```
df.printSchema()
```

```
root
|-- id: integer (nullable = true)
|-- bathrooms: double (nullable = true)
|-- bedrooms: double (nullable = true)
|-- beds: double (nullable = true)
|-- accommodates: integer (nullable = true)
|-- minimum_nights: integer (nullable = true)
|-- number_of_reviews: integer (nullable = true)
|-- review_scores_rating: double (nullable = true)
|-- property_type: string (nullable = true)
|-- room_type: string (nullable = true)
|-- neighborhood: string (nullable = true)
|-- price: double (nullable = true)
```

```
df.select("bathrooms", "bedrooms", "price").show()
```

```
-----+-----+-----+
|bathrooms|bedrooms| price|
-----+-----+-----+
```

```

|      1.0|      1.0|146.65|
|      3.0|      1.0| 252.6|
|      3.0|      1.0|274.16|
|      1.0|      1.0|168.71|
|      4.0|      3.0|431.81|
|      1.0|      1.0|195.22|
|      2.5|      1.0|227.21|
|      1.5|      1.0|259.44|
|      1.0|      1.0|149.63|
|      4.0|      2.0|363.65|
|      3.0|      3.0|429.59|
|      1.0|      4.0|450.65|
|      1.0|      4.0|429.74|
|      4.0|      1.0|319.03|
|      2.0|      4.0|482.72|
|      2.5|      2.0|346.94|
|      1.0|      1.0|252.54|
|      1.0|      2.0|272.25|
|      3.0|      4.0|504.45|
|      1.0|      3.0|362.57|
+-----+-----+-----+

```

only showing top 20 rows

```
df.select("bathrooms", "bedrooms", "price").summary().show()
```

```

+-----+-----+-----+-----+
|summary|      bathrooms|      bedrooms|      price|
+-----+-----+-----+-----+
|  count|           120|           120|           120|
|   mean|2.0791666666666666| 2.1833333333333333|331.68466666666669|
| stddev|1.1320314032919827|1.0999108698292912|99.93166069855937|
|    min|             1.0|             1.0|          144.23|
|   25%|             1.0|             1.0|          259.96|
|   50%|             1.5|             2.0|          318.44|
|   75%|             3.0|             3.0|          408.92|
|    max|             4.0|             4.0|          565.58|
+-----+-----+-----+-----+

```

```
df_clean = df.na.drop(subset=["bathrooms", "bedrooms", "price"])
```

```
df_clean.select("bathrooms", "bedrooms", "price").show()
```

```

+-----+-----+-----+
|bathrooms|bedrooms| price|
+-----+-----+-----+
|      1.0|      1.0|146.65|
|      3.0|      1.0| 252.6|
|      3.0|      1.0|274.16|
|      1.0|      1.0|168.71|
|      4.0|      3.0|431.81|
|      1.0|      1.0|195.22|
|      2.5|      1.0|227.21|
|      1.5|      1.0|259.44|
|      1.0|      1.0|149.63|
|      4.0|      2.0|363.65|
|      3.0|      3.0|429.59|
|      1.0|      4.0|450.65|
|      1.0|      4.0|429.74|
|      4.0|      1.0|319.03|
|      2.0|      4.0|482.72|
|      2.5|      2.0|346.94|
|      1.0|      1.0|252.54|
|      1.0|      2.0|272.25|
|      3.0|      4.0|504.45|
|      1.0|      3.0|362.57|
+-----+-----+-----+

```

only showing top 20 rows

```
trainDF, testDF = df_clean.randomSplit([0.8, 0.2], seed=42)
```

```
print("Training Rows:", trainDF.count())
print("Testing Rows:", testDF.count())
```

```
Training Rows: 98
Testing Rows: 22
```

```
from pyspark.ml.feature import VectorAssembler

vecAssembler = VectorAssembler(
    inputCols=["bathrooms", "bedrooms"],
    outputCol="features"
)
```

```
vecTrainDF = vecAssembler.transform(trainDF)

vecTrainDF.select(
    "bathrooms",
    "bedrooms",
    "features",
    "price"
).show()
```

```
+-----+-----+-----+-----+
|bathrooms|bedrooms| features| price|
+-----+-----+-----+-----+
|      1.0|      1.0|[1.0,1.0]| 146.65|
|      3.0|      1.0|[3.0,1.0]| 252.6|
|      1.0|      1.0|[1.0,1.0]| 168.71|
|      4.0|      3.0|[4.0,3.0]| 431.81|
|      1.0|      1.0|[1.0,1.0]| 195.22|
|      1.5|      1.0|[1.5,1.0]| 259.44|
|      4.0|      2.0|[4.0,2.0]| 363.65|
|      3.0|      3.0|[3.0,3.0]| 429.59|
|      1.0|      4.0|[1.0,4.0]| 450.65|
|      1.0|      4.0|[1.0,4.0]| 429.74|
|      2.0|      4.0|[2.0,4.0]| 482.72|
|      2.5|      2.0|[2.5,2.0]| 346.94|
|      1.0|      1.0|[1.0,1.0]| 252.54|
|      1.0|      2.0|[1.0,2.0]| 272.25|
|      3.0|      4.0|[3.0,4.0]| 504.45|
|      2.5|      1.0|[2.5,1.0]| 262.21|
|      1.5|      2.0|[1.5,2.0]| 283.55|
|      1.0|      4.0|[1.0,4.0]| 447.73|
|      1.0|      1.0|[1.0,1.0]| 199.55|
|      3.0|      2.0|[3.0,2.0]| 335.79|
+-----+-----+-----+-----+
```

only showing top 20 rows

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(
    featuresCol="features",
    labelCol="price",
    predictionCol="prediction"
)
```

```
lrModel = lr.fit(vecTrainDF)

print("Model trained successfully")
```

Model trained successfully

```
predDF = lrModel.transform(vecTrainDF)

predDF.select(
    "bathrooms",
    "bedrooms",
    "features",
    "price",
    "prediction"
).show()
```

```
+-----+-----+-----+-----+-----+
|bathrooms|bedrooms| features| price| prediction|
+-----+-----+-----+-----+-----+
|      1.0|      1.0|[1.0,1.0]| 146.65| 205.81981563880692|
|      3.0|      1.0|[3.0,1.0]| 252.6| 269.0154678055155|
|      1.0|      1.0|[1.0,1.0]| 168.71| 205.81981563880692|
|      4.0|      3.0|[4.0,3.0]| 431.81| 459.4643705517651|
|      1.0|      1.0|[1.0,1.0]| 195.22| 205.81981563880692|
|      1.5|      1.0|[1.5,1.0]| 259.44| 221.61872868048408|
|      4.0|      2.0|[4.0,2.0]| 363.65| 380.0388322031743|
```

3.0	3.0	[3.0,3.0]	429.59	427.8665444684108
1.0	4.0	[1.0,4.0]	450.65	444.0964306331499
1.0	4.0	[1.0,4.0]	429.74	444.0964306331499
2.0	4.0	[2.0,4.0]	482.72	475.6942567165041
2.5	2.0	[2.5,2.0]	346.94	332.642093095286
1.0	1.0	[1.0,1.0]	252.54	205.81981563880692
1.0	2.0	[1.0,2.0]	272.25	285.24535397025454
3.0	4.0	[3.0,4.0]	504.45	507.29208279985846
2.5	1.0	[2.5,1.0]	262.21	253.21655476383833
1.5	2.0	[1.5,2.0]	283.55	301.04426701193177
1.0	4.0	[1.0,4.0]	447.73	444.0964306331499
1.0	1.0	[1.0,1.0]	199.55	205.81981563880692
3.0	2.0	[3.0,2.0]	335.79	348.44100613696315

only showing top 20 rows

```
vecTestDF = vecAssembler.transform(testDF)
```

```
vecTestDF.select(
    "bathrooms",
    "bedrooms",
    "features",
    "price"
).show()
```

bathrooms	bedrooms	features	price
3.0	1.0	[3.0,1.0]	274.16
2.5	1.0	[2.5,1.0]	227.21
1.0	1.0	[1.0,1.0]	149.63
4.0	1.0	[4.0,1.0]	319.03
1.0	3.0	[1.0,3.0]	362.57
4.0	1.0	[4.0,1.0]	255.15
2.0	1.0	[2.0,1.0]	217.86
4.0	3.0	[4.0,3.0]	445.83
3.0	2.0	[3.0,2.0]	336.57
1.0	3.0	[1.0,3.0]	362.99
2.5	2.0	[2.5,2.0]	302.99
4.0	3.0	[4.0,3.0]	452.07
4.0	1.0	[4.0,1.0]	284.39
3.0	4.0	[3.0,4.0]	494.39
1.0	1.0	[1.0,1.0]	169.1
4.0	3.0	[4.0,3.0]	457.96
4.0	2.0	[4.0,2.0]	377.29
1.0	2.0	[1.0,2.0]	272.12
1.0	2.0	[1.0,2.0]	274.42
1.0	2.0	[1.0,2.0]	267.16

only showing top 20 rows

```
predTestDF = lrModel.transform(vecTestDF)
```

```
predTestDF.select(
    "bathrooms",
    "bedrooms",
    "features",
    "price",
    "prediction"
).show()
```

bathrooms	bedrooms	features	price	prediction
3.0	1.0	[3.0,1.0]	274.16	269.0154678055155
2.5	1.0	[2.5,1.0]	227.21	253.21655476383833
1.0	1.0	[1.0,1.0]	149.63	205.81981563880692
4.0	1.0	[4.0,1.0]	319.03	300.61329388886975
1.0	3.0	[1.0,3.0]	362.57	364.6708923017022
4.0	1.0	[4.0,1.0]	255.15	300.61329388886975
2.0	1.0	[2.0,1.0]	217.86	237.41764172216122
4.0	3.0	[4.0,3.0]	445.83	459.4643705517651
3.0	2.0	[3.0,2.0]	336.57	348.44100613696315
1.0	3.0	[1.0,3.0]	362.99	364.6708923017022
2.5	2.0	[2.5,2.0]	302.99	332.642093095286
4.0	3.0	[4.0,3.0]	452.07	459.4643705517651
4.0	1.0	[4.0,1.0]	284.39	300.61329388886975
3.0	4.0	[3.0,4.0]	494.39	507.29208279985846
1.0	1.0	[1.0,1.0]	169.1	205.81981563880692
4.0	3.0	[4.0,3.0]	457.96	459.4643705517651

4.0	2.0	[4.0,2.0]	377.29	380.03883222031743
1.0	2.0	[1.0,2.0]	272.12	285.24535397025454
1.0	2.0	[1.0,2.0]	274.42	285.24535397025454
1.0	2.0	[1.0,2.0]	267.16	285.24535397025454

only showing top 20 rows

```
from pyspark.ml.evaluation import RegressionEvaluator

rmseEvaluator = RegressionEvaluator(
    labelCol="price",
    predictionCol="prediction",
    metricName="rmse"
)

maeEvaluator = RegressionEvaluator(
    labelCol="price",
    predictionCol="prediction",
    metricName="mae"
)

r2Evaluator = RegressionEvaluator(
    labelCol="price",
    predictionCol="prediction",
    metricName="r2"
)

rmse = rmseEvaluator.evaluate(predTestDF)
mae = maeEvaluator.evaluate(predTestDF)
r2 = r2Evaluator.evaluate(predTestDF)

print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

```
RMSE: 22.31320924838175
MAE: 17.4912717458969
R2 Score: 0.9375834729983692
```

```
print("Intercept:", lrModel.intercept)
print("Coefficients:", lrModel.coefficients)
```

```
Intercept: 94.79645122400498
Coefficients: [31.597826083354285, 79.42553833144765]
```

```
extra_features = [
    "bathrooms",
    "bedrooms",
    "beds",
    "accommodates",
    "review_scores_rating"
]

df_extra = df.na.drop(subset=extra_features + ["price"])

trainDF2, testDF2 = df_extra.randomSplit([0.8, 0.2], seed=42)

vecAssembler2 = VectorAssembler(
    inputCols=extra_features,
    outputCol="features"
)

vecTrainDF2 = vecAssembler2.transform(trainDF2)
vecTestDF2 = vecAssembler2.transform(testDF2)

lr2 = LinearRegression(
    featuresCol="features",
    labelCol="price",
    predictionCol="prediction"
)

lrModel2 = lr2.fit(vecTrainDF2)

predTestDF2 = lrModel2.transform(vecTestDF2)
```

```

predTestDF2.select(
    "bathrooms",
    "bedrooms",
    "beds",
    "accommodates",
    "review_scores_rating",
    "features",
    "price",
    "prediction"
).show()

```

bathrooms	bedrooms	beds	accommodates	review_scores_rating	features	price	prediction
3.0	1.0	1.0	4	96.1	[3.0,1.0,1.0,4.0,...]	274.16	300.83402785585554
2.5	1.0	1.0	3	78.0	[2.5,1.0,1.0,3.0,...]	227.21	236.99455295707094
1.0	1.0	1.0	3	72.0	[1.0,1.0,1.0,3.0,...]	149.63	177.42904541647084
4.0	1.0	2.0	4	85.0	[4.0,1.0,2.0,4.0,...]	319.03	316.23405506984807
1.0	3.0	4.0	6	70.2	[1.0,3.0,4.0,6.0,...]	362.57	322.59991992498726
4.0	1.0	1.0	2	77.5	[4.0,1.0,1.0,2.0,...]	255.15	270.3090427818572
2.0	1.0	2.0	4	78.0	[2.0,1.0,2.0,4.0,...]	217.86	238.6605958493364
4.0	3.0	4.0	7	73.4	[4.0,3.0,4.0,7.0,...]	445.83	439.71952496758604
3.0	2.0	2.0	6	73.8	[3.0,2.0,2.0,6.0,...]	336.57	339.2197260319317
1.0	3.0	4.0	7	95.0	[1.0,3.0,4.0,7.0,...]	362.99	382.65501789608265
2.5	2.0	2.0	5	84.2	[2.5,2.0,2.0,5.0,...]	302.99	328.02594989137225
4.0	3.0	3.0	7	88.9	[4.0,3.0,3.0,7.0,...]	452.07	464.7687231141091
4.0	1.0	2.0	2	74.0	[4.0,1.0,2.0,2.0,...]	284.39	267.4264546387973
3.0	4.0	4.0	10	82.4	[3.0,4.0,4.0,10.0,...]	494.39	514.2630933994375
1.0	1.0	1.0	2	74.6	[1.0,1.0,1.0,2.0,...]	169.1	167.98770695328312
4.0	3.0	4.0	8	70.9	[4.0,3.0,4.0,8.0,...]	457.96	449.3455851808026
4.0	2.0	2.0	5	71.5	[4.0,2.0,2.0,5.0,...]	377.29	353.04849017657557
1.0	2.0	3.0	6	79.5	[1.0,2.0,3.0,6.0,...]	272.12	288.6886021730355
1.0	2.0	3.0	4	90.2	[1.0,2.0,3.0,4.0,...]	274.42	279.9656214982474
1.0	2.0	3.0	4	82.2	[1.0,2.0,3.0,4.0,...]	267.16	265.18788149593854

only showing top 20 rows

```

rmse2 = rmseEvaluator.evaluate(predTestDF2)
mae2 = maeEvaluator.evaluate(predTestDF2)
r2_2 = r2Evaluator.evaluate(predTestDF2)

```

```

print("Original Model RMSE:", rmse)
print("Original Model MAE:", mae)
print("Original Model R2:", r2)

```

```

print("Improved Model RMSE:", rmse2)
print("Improved Model MAE:", mae2)
print("Improved Model R2:", r2_2)

```

```

Original Model RMSE: 22.31320924838175
Original Model MAE: 17.4912717458969
Original Model R2: 0.9375834729983692
Improved Model RMSE: 17.663806494054135
Improved Model MAE: 14.363809703970889
Improved Model R2: 0.9608849292314982

```